

LaraClassifier - Kompletna Override Strategija

Kako da override-uješ **SVE** u LaraClassifier-u (Controllers, Models, Middleware, Commands, itd.) bez diranja originalnih fajlova.

1. Controllers Override

Metoda A: Route Override (NAJJEDNOSTAVNIJE)

Registruješ svoje rute **POSLE** LaraClassifier ruta - Laravel koristi **prvu matching rutu**.

```
php

// Modules/MojeCustomFunkcije/Providers/MojeCustomFunkcijeServiceProvider.php

public function boot()
{
    // Ovo mora biti NAKON što se učitaju LaraClassifier rute
    $this->app->booted(function () {
        $this->overrideRoutes();
    });
}

protected function overrideRoutes()
{
    // Override specifične rute
    Route::get('/posts/{id}', [\Modules\MojeCustomFunkcije\Http\Controllers\CustomPostController::class, 'show']->name('posts.show'));

    Route::get('/home', [\Modules\MojeCustomFunkcije\Http\Controllers\CustomHomeController::class, 'index']->name('home'));
}
```

Struktura:

```
Modules/MojeCustomFunkcije/
├── Http/
│   ├── Controllers/
│   │   ├── CustomPostController.php
│   │   └── CustomHomeController.php
```

Primer Custom Controllera:

php

```
// Modules/MojeCustomFunkcije/Http/Controllers/CustomPostController.php
```

```
namespace Modules\MojeCustomFunkcije\Http\Controllers;
```

```
use App\Http\Controllers\PostController; // Original
```

```
use Illuminate\Http\Request;
```

```
class CustomPostController extends PostController
```

```
{
```

```
    /**
```

```
     * Override samo metode koje želiš
```

```
     */
```

```
    public function show($id)
```

```
    {
```

```
        // Tvoja custom logika
```

```
        $post = $this->findPostWithCustomLogic($id);
```

```
        // Možeš pozvati parent metodu ako želiš
```

```
        // return parent::show($id);
```

```
        return view('mojecustomfunkcije::posts.show', compact('post'));
```

```
    }
```

```
    /**
```

```
     * Dodaj potpuno nove metode
```

```
     */
```

```
    public function myCustomAction(Request $request)
```

```
    {
```

```
        // Nova funkcionalnost
```

```
    }
```

```
    /**
```

```
     * Ostale metode (index, create, edit...) ostaju iz parent klase
```

```
     */
```

```
}
```

Metoda B: Container Binding Override

Override-uješ ceo controller preko Laravel Service Container:

php

// Modules/MojeCustomFunkcije/Providers/MojeCustomFunkcijeServiceProvider.php

```
public function register()
{
    // Bind-uj svoj controller umesto originalnog
    $this->app->bind(
        \App\Http\Controllers\PostController::class,
        \Modules\MojeCustomFunkcije\Http\Controllers\CustomPostController::class
    );

    $this->app->bind(
        \App\Http\Controllers\HomeController::class,
        \Modules\MojeCustomFunkcije\Http\Controllers\CustomHomeController::class
    );
}
```

Prednost: Sve rute koje koriste `PostController` će automatski koristiti tvoj `CustomPostController`.

2. Models Override

Metoda A: Extend + Container Binding

php

// Modules/MojeCustomFunkcije/Entities/CustomPost.php

namespace Modules\MojeCustomFunkcije\Entities;

use App\Models\Post;

class CustomPost extends Post

{

/**

* Override postojeće metode

*/

public function getFormattedPriceAttribute()

{

// Tvoja custom logika

return number_format(\$this->price, 2) . ' RSD';

}

/**

* Dodaj nove relationships

*/

public function customFeatures()

{

return \$this->hasMany(CustomFeature::class);

}

/**

* Override scopes

*/

public function scopeActive(\$query)

{

return \$query->where('status', 'active')

->where('custom_field', true); // Tvoja custom logika

}

/**

* Dodaj nove metode

*/

public function myCustomMethod()

{

// Nova funkcionalnost

}

}

Registruj binding:

php

// Service Provider

```
public function register()
{
    // Kada god neko traži Post, dobija CustomPost
    $this->app->bind(
        \App\Models\Post::class,
        \Modules\MojeCustomFunkcije\Entities\CustomPost::class
    );
}
```

Metoda B: Model Events za Injection

Ako ne želiš full override, koristi model events:

php

// Service Provider

```
public function boot()
{
    // Inject custom behavior u postojeći model
    \App\Models\Post::retrieved(function ($post) {
        $post->custom_data = $this->getCustomData($post);
    });

    \App\Models\Post::creating(function ($post) {
        // Custom logika pre kreiranja
        $post->custom_field = 'default_value';
    });

    \App\Models\Post::created(function ($post) {
        // Custom logika nakon kreiranja
        $this->sendCustomNotification($post);
    });
}
```

3. Middleware Override

Registracija Custom Middleware

php

```
// Modules/MojeCustomFunkcije/Http/Middleware/CustomAuthMiddleware.php
```

```
namespace Modules\MojeCustomFunkcije\Http\Middleware;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
class CustomAuthMiddleware
```

```
{
```

```
    public function handle(Request $request, Closure $next)
```

```
    {
```

```
        // Tvoja custom auth logika
```

```
        if (!$this->customAuthCheck($request)) {
```

```
            return redirect('/custom-login');
```

```
        }
```

```
        return $next($request);
```

```
    }
```

```
    protected function customAuthCheck($request)
```

```
    {
```

```
        // Tvoja logika
```

```
        return true;
```

```
    }
```

```
}
```

Registruj u Service Provider:

php

```
public function boot()
```

```
{
```

```
    // Dodaj novi middleware
```

```
    $this->app['router']->aliasMiddleware('custom.auth', CustomAuthMiddleware::class);
```

```
    // Override postojeći middleware alias
```

```
    $this->app['router']->aliasMiddleware('auth', CustomAuthMiddleware::class);
```

```
    // Ili dodaj u middleware grupu
```

```
    $this->app['router']->pushMiddlewareToGroup('web', CustomAuthMiddleware::class);
```

```
}
```

4. Config Override

Metoda: Merge ili Replace Config

```
php

// Modules/MojeCustomFunkcije/Config/custom-settings.php

return [
    'feature_enabled' => true,
    'custom_option' => 'value',
];
```

U Service Provider:

```
php

public function register()
{
    // Merge sa postojećim config
    $this->mergeConfigFrom(
        module_path('MojeCustomFunkcije', 'Config/custom-settings.php'),
        'app' // Ime config fajla
    );

    // Ili potpuno override-uj config key
    config(['app.name' => 'Moj Custom Naziv']);
    config(['lاراclassifier.setting' => 'custom_value']);
}
```

Pristup config vrednostima:

```
php

config('app.custom_option'); // 'value'
```

5. Commands (Artisan) Override

Custom Command

php

// Modules/MojeCustomFunkcije/Console/CustomCommand.php

```
namespace Modules\MojeCustomFunkcije\Console;
```

```
use Illuminate\Console\Command;
```

```
class CustomCommand extends Command
```

```
{
```

```
    protected $signature = 'custom:process [--option=]';
```

```
    protected $description = 'Custom processing command';
```

```
    public function handle()
```

```
    {
```

```
        $this->info('Running custom command...');
```

```
        // Tvoja logika
```

```
        $this->info('Done!');
```

```
    }
```

```
}
```

Registruj:

php

// Service Provider

```
protected $commands = [
```

```
    \Modules\MojeCustomFunkcije\Console\CustomCommand::class,
```

```
];
```

```
public function boot()
```

```
{
```

```
    if ($this->app->runningInConsole()) {
```

```
        $this->commands($this->commands);
```

```
    }
```

```
}
```

Override postojeći command:

php

```
public function boot()
{
    // Unregister originalni command
    unset(Artisan::all()['original:command']);

    // Registruj svoj
    $this->commands([CustomCommand::class]);
}
```

6. Policies Override

Custom Policy

php

// Modules/MojeCustomFunkcije/Policies/CustomPostPolicy.php

```
namespace Modules\MojeCustomFunkcije\Policies;
```

```
use App\Models\User;
```

```
use App\Models\Post;
```

```
class CustomPostPolicy
```

```
{
    public function view(User $user, Post $post)
    {
        // Tvoja custom logika
        return $user->hasCustomPermission('view-posts');
    }
}
```

```
public function update(User $user, Post $post)
```

```
{
    return $user->id === $post->user_id
        || $user->hasCustomRole('moderator');
}
```

```
public function customAction(User $user, Post $post)
```

```
{
    // Potpuno nova policy metoda
    return $user->custom_field === 'allowed';
}
```

Registruj:

php

// Service Provider

```
use Illuminate\Support\Facades\Gate;
use App\Models\Post;
use Modules\MojeCustomFunkcije\Policies\CustomPostPolicy;

public function boot()
{
    // Override postojeću policy
    Gate::policy(Post::class, CustomPostPolicy::class);
}
```

7. Requests (Form Validation) Override

Custom Request

php

// Modules/MojeCustomFunkcije/Http/Requests/CustomPostRequest.php

namespace Modules\MojeCustomFunkcije\Http\Requests;

use App\Http\Requests\PostRequest; // Original

class CustomPostRequest extends PostRequest

{

public function rules()

{

// Extend original rules

\$rules = parent::rules();

// Dodaj custom rules

\$rules['custom_field'] = 'required|string|max:255';

\$rules['title'] = 'required|string|max:500'; // Override existing

return \$rules;

}

public function messages()

{

return [

'custom_field.required' => 'Custom field je obavezan!',

];

}

/**

* Custom validation logic

*/

public function withValidator(\$validator)

{

\$validator->after(function (\$validator) {

if (\$this->somethingElseIsInvalid()) {

\$validator->errors()->add('field', 'Something is wrong!');

}

});

}

}

Koristi u controlleru:

php

```
public function store(CustomPostRequest $request)
{
    // $request je već validiran sa tvojim custom rules
    $validated = $request->validated();
}
```

8. Events & Listeners Override

Custom Event

php

// Modules/MojeCustomFunkcije/Events/CustomPostCreated.php

```
namespace Modules\MojeCustomFunkcije\Events;
```

```
use App\Models\Post;
```

```
use Illuminate\Foundation\Events\Dispatchable;
```

```
use Illuminate\Queue\SerializesModels;
```

```
class CustomPostCreated
```

```
{
```

```
    use Dispatchable, SerializesModels;
```

```
    public $post;
```

```
    public $customData;
```

```
    public function __construct(Post $post, array $customData = [])
```

```
    {
```

```
        $this->post = $post;
```

```
        $this->customData = $customData;
```

```
    }
```

```
}
```

Custom Listener

php

// Modules/MojeCustomFunkcije/Listeners/SendCustomNotification.php

namespace Modules\MojeCustomFunkcije\Listeners;

use Modules\MojeCustomFunkcije\Events\CustomPostCreated;

class SendCustomNotification

{

public function handle(CustomPostCreated \$event)

{

// Tvoja custom notification logika

\$post = \$event->post;

// Send notification, log, whatever...

}

}

Registruj:

php

// Service Provider

```
protected $listen = [  
    \Modules\MojeCustomFunkcije\Events\CustomPostCreated::class => [  
        \Modules\MojeCustomFunkcije\Listeners\SendCustomNotification::class,  
    ],  
  
    // Override postojeći listener  
    \App\Events\PostCreated::class => [  
        \Modules\MojeCustomFunkcije\Listeners\CustomPostCreatedListener::class,  
    ],  
];  
  
public function boot()  
{  
    // Ili hook-uj se na postojeći event  
    \Illuminate\Support\Facades\Event::listen(  
        \App\Events\PostCreated::class,  
        function ($event) {  
            // Tvoja dodatna logika  
        }  
    );  
}
```

9. Repositories Override (Ako LaraClassifier Koristi Repository Pattern)

Custom Repository

php

// Modules/MojeCustomFunkcije/Repositories/CustomPostRepository.php

namespace Modules\MojeCustomFunkcije\Repositories;

use App\Repositories\PostRepository;

use Modules\MojeCustomFunkcije\Entities\CustomPost;

class CustomPostRepository extends PostRepository

{

public function __construct()

{

// Koristi tvoj custom model

\$this->model = new CustomPost();

}

/**

** Override postojeću metodu*

**/*

public function findActive()

{

return *\$this->model*

->where('status', 'active')

->where('custom_field', true) // Tvoja custom logika

->get();

}

/**

** Nova metoda*

**/*

public function findByCustomCriteria(\$criteria)

{

// Tvoja logika

}

}

Bind u Service Provider:

php

```
public function register()
{
    $this->app->bind(
        \App\Repositories\PostRepository::class,
        \Modules\MojeCustomFunkcije\Repositories\CustomPostRepository::class
    );
}
```

10. Services Override

Custom Service

php

```
// Modules/MojeCustomFunkcije/Services/CustomPostService.php
```

```
namespace Modules\MojeCustomFunkcije\Services;
```

```
use App\Services\PostService;
```

```
class CustomPostService extends PostService
```

```
{  
    /**  
     * Override processing logic  
     */  
    public function processPost($data)  
    {  
        // Pre-processing  
        $data = $this->customPreProcessing($data);  
  
        // Call parent ili full custom  
        $result = parent::processPost($data);  
  
        // Post-processing  
        $this->customPostProcessing($result);  
  
        return $result;  
    }  
  
    protected function customPreProcessing($data)  
    {  
        // Tvoja logika  
        return $data;  
    }  
  
    /**  
     * Nova service metoda  
     */  
    public function myCustomService()  
    {  
        // Nova funkcionalnost  
    }  
}
```

Bind:

php

```
public function register()
{
    $this->app->singleton(
        \App\Services\PostService::class,
        \Modules\MojeCustomFunkcije\Services\CustomPostService::class
    );
}
```

11. Translations Override

Custom Translation Fajlovi

```
Modules/MojeCustomFunkcije/
├── Resources/
│   ├── lang/
│   │   ├── en/
│   │   │   ├── messages.php
│   │   │   └── validation.php
│   └── sr/
│       ├── messages.php
│       └── validation.php
```

Registruj:

php

// Service Provider

```
public function boot()
{
    $this->loadTranslationsFrom(
        module_path('MojeCustomFunkcije', 'Resources/lang'),
        'mojecustomfunkcije'
    );

    // Override postojeće translations
    $this->loadTranslationsFrom(
        module_path('MojeCustomFunkcije', 'Resources/lang'),
        'laraclassifier'
    );
}
```

Korišćenje:

php

```
__('mojecustomfunkcije::messages.welcome');  
trans('mojecustomfunkcije::validation.required');
```

12. Migrations Override/Extend

Dodaj Nove Kolone

php

```
// Modules/MojeCustomFunkcije/Database/Migrations/2024_01_01_000000_add_custom_fields_to_posts.php
```

```
use Illuminate\Database\Migrations\Migration;  
use Illuminate\Database\Schema\Blueprint;  
use Illuminate\Support\Facades\Schema;
```

```
class AddCustomFieldsToPosts extends Migration  
{  
    public function up()  
    {  
        Schema::table('posts', function (Blueprint $table) {  
            $table->string('custom_field')->nullable();  
            $table->boolean('is_featured')->default(false);  
            $table->json('custom_data')->nullable();  
        });  
    }  
  
    public function down()  
    {  
        Schema::table('posts', function (Blueprint $table) {  
            $table->dropColumn(['custom_field', 'is_featured', 'custom_data']);  
        });  
    }  
}
```

Registruj:

php

// Service Provider

```
public function boot()
{
    $this->loadMigrationsFrom(
        module_path('MojeCustomFunkcije', 'Database/Migrations')
    );
}
```

13. API Resources Override

Custom Resource

php

// Modules/MojeCustomFunkcije/Http/Resources/CustomPostResource.php

```
namespace Modules\MojeCustomFunkcije\Http\Resources;
```

```
use App\Http\Resources\PostResource;
```

```
class CustomPostResource extends PostResource
```

```
{
```

```
    public function toArray($request)
```

```
    {
```

```
        // Get parent data
```

```
        $data = parent::toArray($request);
```

```
        // Add custom fields
```

```
        $data['custom_field'] = $this->custom_field;
```

```
        $data['custom_computed'] = $this->computeCustomValue();
```

```
        // Remove fields
```

```
        unset($data['internal_field']);
```

```
        return $data;
```

```
    }
```

```
    protected function computeCustomValue()
```

```
    {
```

```
        // Tvoja logika
```

```
        return 'computed_value';
```

```
    }
```

```
}
```

Kompletan Primer Service Provider-a

Evo kako izgleda kompletan Service Provider sa svim override-ima:

php

// Modules/MojeCustomFunkcije/Providers/MojeCustomFunkcijeServiceProvider.php

namespace Modules\MojeCustomFunkcije\Providers;

use Illuminate\Support\ServiceProvider;

use Illuminate\Support\Facades\View;

use Illuminate\Support\Facades\Gate;

use Illuminate\Support\Facades\Route;

class MojeCustomFunkcijeServiceProvider extends ServiceProvider

{

protected \$listen = [

 \App\Events\PostCreated::class => [

 \Modules\MojeCustomFunkcije\Listeners\CustomPostCreatedListener::class,

],

];

/**

* Register bindings

*/

public function register()

{

 // Config

 \$this->mergeConfigFrom(

 module_path('MojeCustomFunkcije', 'Config/config.php'),

 'mojecustomfunkcije'

);

 // Controllers

 \$this->app->bind(

 \App\Http\Controllers\PostController::class,

 \Modules\MojeCustomFunkcije\Http\Controllers\CustomPostController::class

);

 // Models

 \$this->app->bind(

 \App\Models\Post::class,

 \Modules\MojeCustomFunkcije\Entities\CustomPost::class

);

 // Repositories

 \$this->app->bind(

 \App\Repositories\PostRepository::class,

 \Modules\MojeCustomFunkcije\Repositories\CustomPostRepository::class

);

```

);

// Services
$this->app->singleton(
    \App\Services\PostService::class,
    \Modules\MojeCustomFunkcije\Services\CustomPostService::class
);
}

/**
 * Bootstrap services
 */
public function boot()
{
    // Views
    $this->overrideViews();
    $this->loadViewsFrom(module_path('MojeCustomFunkcije', 'Resources/views'), 'mojecustomfunkcije');

    // Routes
    $this->loadRoutesFrom(module_path('MojeCustomFunkcije', 'Routes/web.php'));
    $this->app->booted(function () {
        $this->overrideRoutes();
    });

    // Migrations
    $this->loadMigrationsFrom(module_path('MojeCustomFunkcije', 'Database/Migrations'));

    // Translations
    $this->loadTranslationsFrom(module_path('MojeCustomFunkcije', 'Resources/lang'), 'mojecustomfunkcije');

    // Middleware
    $this->registerMiddleware();

    // Policies
    $this->registerPolicies();

    // Commands
    if ($this->app->runningInConsole()) {
        $this->commands([
            \Modules\MojeCustomFunkcije\Console\CustomCommand::class,
        ]);
    }

    // Model Events
    $this->registerModelEvents();
}

```

```

protected function overrideViews()
{
    View::getFinder()->prependLocation(
        module_path('MojeCustomFunkcije', 'Resources/views/overrides')
    );
}

protected function overrideRoutes()
{
    Route::get('/posts/{id}', [
        \Modules\MojeCustomFunkcije\Http\Controllers\CustomPostController::class,
        'show'
    ])->name('posts.show');
}

protected function registerMiddleware()
{
    $this->app['router']->aliasMiddleware(
        'custom.auth',
        \Modules\MojeCustomFunkcije\Http\Middleware\CustomAuthMiddleware::class
    );
}

protected function registerPolicies()
{
    Gate::policy(
        \App\Models\Post::class,
        \Modules\MojeCustomFunkcije\Policies\CustomPostPolicy::class
    );
}

protected function registerModelEvents()
{
    \App\Models\Post::creating(function ($post) {
        $post->custom_field = 'default';
    });
}
}

```

Checklist: Šta Možeš Da Override-uješ

-  **Views** - `View::getFinder()->prependLocation()`
 -  **Controllers** - Route override ili Container binding
 -  **Models** - Extend + Container binding
 -  **Middleware** - `aliasMiddleware()` ili `pushMiddlewareToGroup()`
 -  **Config** - `mergeConfigFrom()` ili `config()`
 -  **Commands** - Registruj svoje, unregister stare
 -  **Policies** - `Gate::policy()`
 -  **Requests** - Extend i koristi u controlleru
 -  **Events/Listeners** - Registruj u `$listen` property
 -  **Repositories** - Container binding
 -  **Services** - Container binding
 -  **Translations** - `loadTranslationsFrom()`
 -  **Migrations** - `loadMigrationsFrom()` (extend tabele)
 -  **API Resources** - Extend i override `toArray()`
-

Dokumentacija Svih Override-a

Preporučujem da napraviš:

markdown

```
<!-- Modules/MojeCustomFunkcije/OVERRIDES.md -->
```

Custom Overrides

Controllers

- PostController → CustomPostController
- Razlog: Custom search logika
- Override metode: show(), index()

Models

- Post → CustomPost
- Razlog: Dodati custom fields i relationships
- Nove kolone: custom_field, is_featured

Views

- layouts/inc/header.blade.php
- home/index.blade.php

Services

- PostService → CustomPostService
- Razlog: Custom processing logic

Config

- app.name = "Moj Custom Naziv"
- laraclassifier.setting = "custom_value"

Best Practices

- ✓ **Extend umesto potpunog override-a** gde god je moguće
- ✓ **Dokumentuj sve što si override-ovao**
- ✓ **Koristi Container binding za klase, route override za controller akcije**
- ✓ **Testuj nakon svakog LaraClassifier update-a**
- ✓ **Čuvaj originalnu strukturu i namespace**
- ✗ **NE edituj originalne LaraClassifier fajlove**
- ✗ **NE override-uj što ne moraš**
- ✗ **NE zaboravi da update-uješ dokumentaciju**